

Cross-Traffic Intersection Management for Autonomous Vehicles

Faysal Ahammed Tonmoy, MD Jubair Salehin Razin, MD Rafiul Islam, MD Azadul Islam
Embedded Electronic Engineering Lab A

Team A · Group 01

Abstract—This paper presents the design of a Road-Side-Unit (RSU) based autonomous intersection controller that replaces fixed traffic signals with an on-demand grant scheme for co-operative vehicles. Starting from a set of functional and non-functional requirements, the controller is specified as a network of UPPAAL timed automata and formally verified against safety and reachability queries, including collision freedom and deadlock freedom. The verified model is then mapped into a deterministic C baseline and then into a hardware/software codesign proposal. The software part keeps scheduling, messaging, and priority logic in FreeRTOS, while the repeated conflict checking is proposed as a VHDL safety coprocessor. The system supports both M/M/1 and M/M/2 crossing service, emergency pre-emption, V2I/I2V/V2V communication, and fairness logic for avoiding starvation.

Index Terms—RSU, hardware/software codesign, FreeRTOS, VHDL, UPPAAL, autonomous intersection, M/M/1, M/M/2, emergency priority, formal verification.

I. INTRODUCTION

Autonomous and cooperative vehicles do not need a fixed-cycle traffic light to coordinate an intersection. A Road-Side Unit (RSU) can grant crossing permission on demand and react to the actual traffic pattern. The central engineering problem is that this dynamic grant scheme must remain safe: the RSU has to avoid collisions, resolve priority between emergency and normal traffic, and behave predictably enough to be verified.

This paper compares two service disciplines for the crossing zone. The first is a conservative M/M/1 model where only one vehicle may occupy the intersection at a time. The second is an M/M/2 model where two non-conflicting vehicles may cross simultaneously. The project follows a model-based path: requirements first, then UPPAAL verification, then a deterministic C baseline, and finally a hardware/software codesign proposal using FreeRTOS and a VHDL safety coprocessor.

The contribution of this paper is:

- a functional and non-functional requirements specification for the RSU controller;
- UPPAAL timed-automata models of the controller and vehicle fleet;
- comparison of a single-queue and a four-queue UPPAAL model;
- a quantitative comparison of waiting time and crossing throughput for M/M/1 and M/M/2 service;
- a deterministic C baseline based on the verified model;
- a hardware/software partitioning proposal with FreeRTOS software and VHDL conflict checking;

- validation of M/M/1, M/M/2, emergency priority, and safety behaviour.

II. RELATED WORK

Reservation-based control of intersections for autonomous vehicles was popularised by Dresner and Stone, whose Autonomous Intersection Management (AIM) scheme lets vehicles reserve space-time slots from a central manager instead of obeying fixed signals [6]. Later cooperative intersection management approaches, surveyed by Chen and Englund, extend this idea with vehicle-to-infrastructure and vehicle-to-vehicle communication and with a range of scheduling and priority policies [7]. The RSU grant scheme in this paper follows the same reservation-style principle: a vehicle may enter the conflict zone only once the infrastructure has granted it.

Most AIM and cooperative-intersection studies evaluate throughput and delay through simulation, but comparatively few provide a formal safety guarantee for the admission logic itself. Formal verification of such timing-dependent controllers builds on the theory of timed automata introduced by Alur and Dill [8], for which the UPPAAL model checker offers an established modelling and verification environment [9]. This work combines the two directions: the reservation-style RSU controller is expressed as a network of UPPAAL timed automata and formally checked for collision and deadlock freedom before being mapped to a deterministic C baseline and a hardware/software codesign proposal. The single-queue and four-queue variants studied here therefore contribute a formally verified, per-approach view of the intersection admission problem rather than a purely simulation-based one.

III. REQUIREMENTS AND SYSTEM SCOPE

The system is an RSU-based autonomous intersection controller. Vehicles arrive from North, South, East, and West. Normal vehicles communicate with the RSU using V2I messages such as request, grant, wait, and done. Emergency vehicles additionally broadcast a V2V emergency beacon to warn nearby vehicles. The RSU manages waiting vehicles, checks path conflicts, grants or withholds crossing permission, and prevents unsafe simultaneous movement.

Figure 1 shows the main actors of the system: normal vehicles, emergency vehicles, the RSU controller, and a simulation or monitoring component.

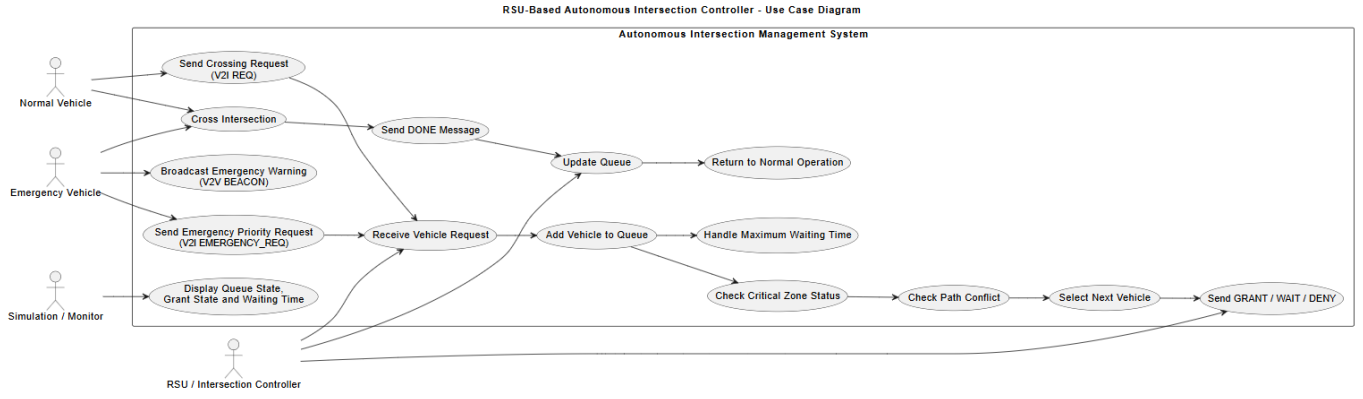


Fig. 1. Use-case diagram of the RSU-based autonomous intersection management system. Normal and emergency vehicles interact with the RSU through V2I and V2V messages. The RSU checks requests, conflicts, priority, and crossing permission, while the monitor observes queue state, grant state, and waiting time.

TABLE I
MAIN FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

ID	Type	Requirement
FR2	Functional	Vehicle sends a V2I crossing request to the RSU
FR8	Functional	RSU checks whether vehicle paths conflict
FR9	Functional	RSU sends GRANT only when the vehicle can safely enter
FR15	Functional	System supports emergency vehicle priority
NFR1	Safety	System is collision-free in the critical zone
NFR3	Safety	System is deadlock-free
NFR4	Fairness	System prevents starvation
NFR6	Priority	Emergency requests are handled with higher priority
NFR10	Determinism	System behaves deterministically
NFR18	Comparison	System supports M/M/1 vs. M/M/2 comparison

Table I lists the main functional and non-functional requirements used in the design.

IV. SYSTEM SCENARIO AND TRAFFIC MODEL

The main street runs East–West and the side street runs North–South. The RSU controls the shared conflict zone at the intersection. The verified scenario instantiates six vehicles, summarized in Table II. Vehicle V0 is the emergency vehicle. V1 and V2 are the main-road pair used to show non-conflicting parallel crossing in M/M/2 mode. V4 and V5 form the critical conflict pair: North-right and South-left.

Figure 2 shows the queueing idea. In M/M/1 mode only one vehicle may cross at a time. In M/M/2 mode two vehicles may cross at the same time only if their movements do not conflict.¹

¹In this project the vehicle arrivals are periodic and the crossing times are fixed, so in Kendall’s notation the process is closer to D/D/1 and D/D/2 (deterministic arrivals and deterministic service). The labels M/M/1 and M/M/2 are used throughout in the looser sense of a single-server versus a two-server crossing, following the project convention, and are not intended to assert Markovian (memoryless) arrival or service distributions.

TABLE II
VEHICLE FLEET AS INSTANTIATED IN THE VERIFIED UPPAAL SYSTEM BLOCK

Vehicle	Origin	Movement	Emergency	Period	Role
V0	East	Straight	Yes	30	Emergency vehicle on main street
V1	East	Straight	No	6	Main street, M/M/2 overlap partner
V2	West	Straight	No	8	Main street, opposite direction
V3	North	Straight	No	10	Side street, straight
V4	North	Right	No	12	Critical right-turn vehicle
V5	South	Left	No	12	Critical left-turn vehicle

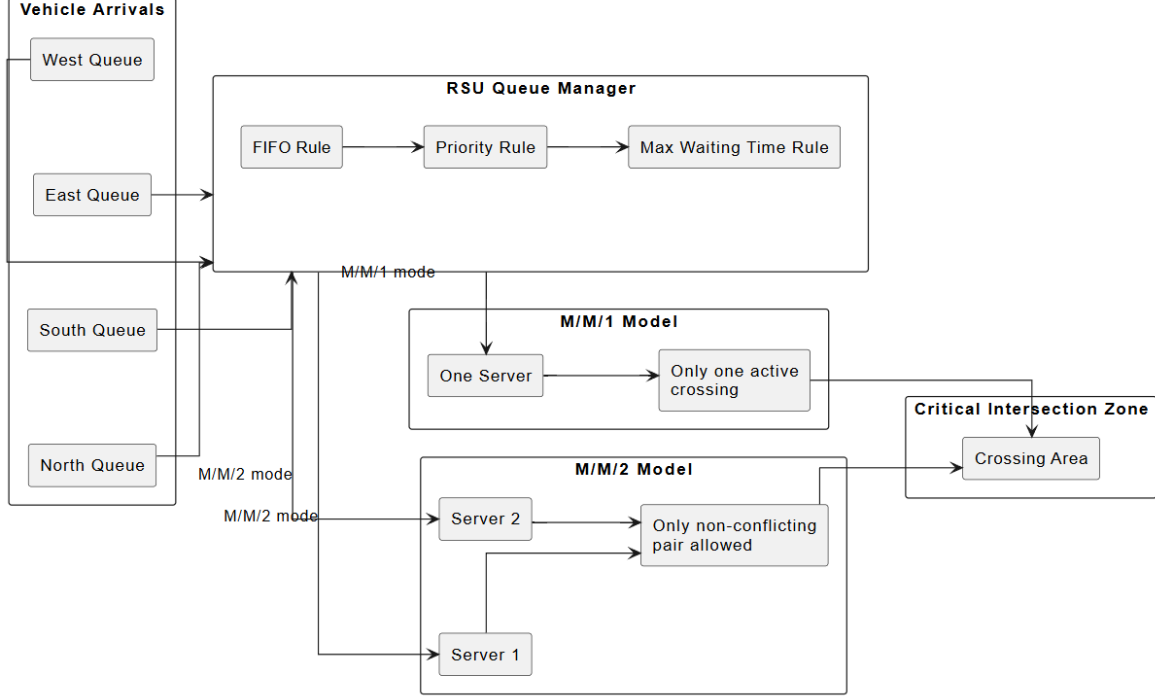


Fig. 2. M/M/1 and M/M/2 queueing model of the intersection. The RSU queue manager admits either one vehicle into the critical zone in M/M/1 mode, or up to two non-conflicting vehicles in M/M/2 mode.

V. SYSTEM ARCHITECTURE

Figure 3 shows the layered system architecture. Vehicles communicate with the RSU using V2I and V2V messages. Inside the RSU, the communication manager, queue manager, emergency handler, conflict checker, scheduler, timer manager, and grant manager together decide whether a vehicle may enter the intersection.

The vehicle tasks V0–V5 generate requests and react to grants. The RSU controller owns the admission decision. FreeRTOS queues carry V2I, I2V, and V2V messages. A shared state region holds arrays such as `waiting[]`, `crossing[]`, `origin[]`, and `turn[]`. The safety coprocessor reads this state and returns `safe_to_grant`.

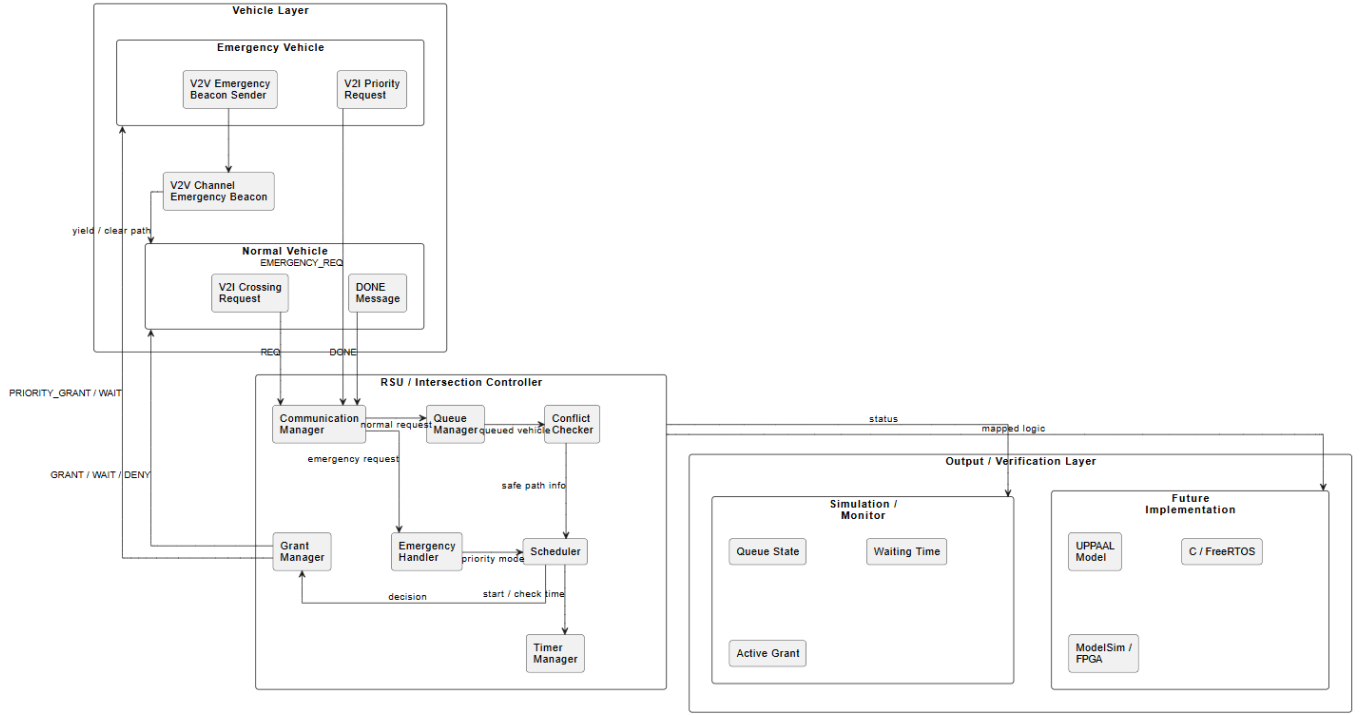


Fig. 3. Layered system architecture. Vehicles communicate with the RSU controller over V2I and V2V channels. The RSU internal blocks are mapped to the UPPAAL model, C/FreeRTOS implementation, and VHDL/FPGA safety coprocessor.

VI. FORMAL MODEL AND ABSTRACT BASELINE

The controller was first specified in UPPAAL as a network of timed automata with two main templates: *Vehicle* and *RSU_Controller*. The vehicle automaton waits for its arrival period, sends a request, waits for a grant, enters the crossing zone, and leaves after the crossing time. The RSU controller receives requests, updates queues, checks emergency priority, checks safety, and grants crossing permission.

A. UPPAAL Model Variants

Two UPPAAL variants were used during modelling. The first variant uses four separate approach queues, one queue for each incoming road direction. The second variant uses a simpler single waiting structure for the six-vehicle intersection model. Both models keep the same safety idea: the RSU may grant a vehicle only when the intersection capacity is not exceeded and no conflicting vehicle is already crossing. The comparison is useful because the single-queue model is easier to read and verify first, while the four-queue model is closer to the real intersection structure where vehicles arrive from East, West, North, and South approaches.

1) *Four-Queue UPPAAL Model*: The four-queue model represents one queue per approach: East, West, North, and South. Each vehicle first sends a V2I request and is inserted into the queue of its origin road. The RSU then considers the head vehicle of each approach queue and selects the globally best grantable candidate according to emergency priority, main-street priority, turning priority, and the conflict

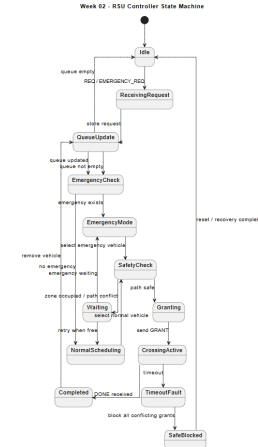


Fig. 4. RSU controller state machine underlying the verified UPPAAL model and its deterministic C mapping.

check. The physical intersection is still shared: therefore, the four queues do not mean four independent servers. In M/M/1 mode only one vehicle can occupy the intersection, while in M/M/2 mode at most two non-conflicting vehicles can cross together.

2) *Single-Queue UPPAAL Model*: The single-queue model is the simpler baseline model. It keeps the same six vehicles, the same emergency-priority concept, the same M/M/1 and M/M/2 service modes, and the same conflict rules. However, it does not separate waiting vehicles into four road-specific

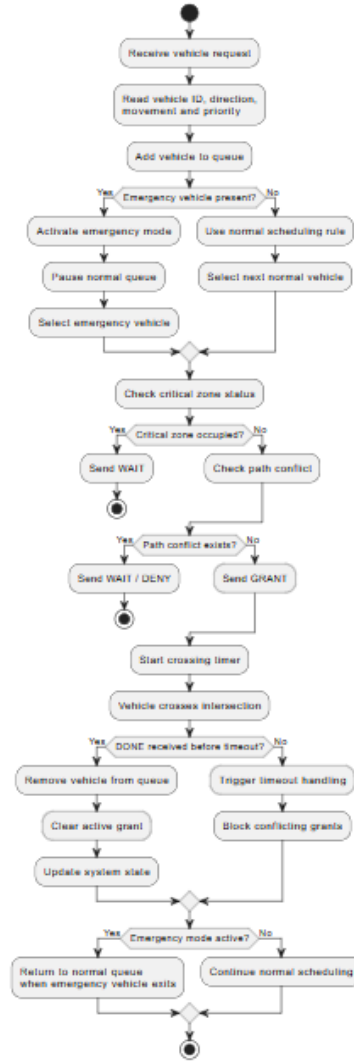


Fig. 5. Activity flow of the RSU controller from receiving vehicle requests to checking emergency priority, updating queues, performing safety checks, granting access, and completing the crossing.

queues. Instead, the controller checks the waiting vehicles globally and grants the best safe candidate according to priority and conflict functions. This model was useful as the first abstract verification step because it focuses on the core admission decision.

3) *Summary and Comparison of the Two Queueing Models:* Two variants of the RSU-controlled intersection were modelled and verified in UPPAAL. Both share the same underlying safety, priority, and conflict logic, and differ only in how waiting vehicles are organised before being granted access to the intersection.

In the single-queue model, all approaching vehicles are managed in one shared priority queue at the RSU. The RSU evaluates every waiting vehicle globally and grants access to the highest-priority vehicle for which intersection capacity is available. Priority follows emergency, then main road, then side road, with right-turn-before-left-turn resolving conflicting manoeuvres and vehicle ID as a tie-breaker. This corresponds

to a single pooled queueing station, operating either in M/M/1 or M/M/2 mode.

In the four-queue model, waiting vehicles are partitioned by their approach into four separate queues: East, West, North, and South. This is represented by the array `queueLen[R]` with $R = 4$. Service follows a head-of-line discipline: only the highest-priority vehicle of each approach, namely its queue head computed by `headOfQueue`, is eligible for service. The RSU then selects the globally best grantable head across the four approaches. This corresponds to a per-approach controller whose four queues feed a single shared crossing.

In both models, intersection capacity and conflict detection remain intersection-wide. At most one vehicle in M/M/1 mode, or two non-conflicting vehicles in M/M/2 mode, may occupy the crossing simultaneously, because the physical intersection is a single shared resource regardless of how the waiting vehicles are organised. The four queues therefore share one server; they are not four independent servers.



Fig. 6. UPPAAL four-queue model. The screenshots show the vehicle/controller automata and the approach-queue based RSU logic. Each road direction has its own queue, but all queues feed the same shared intersection capacity.

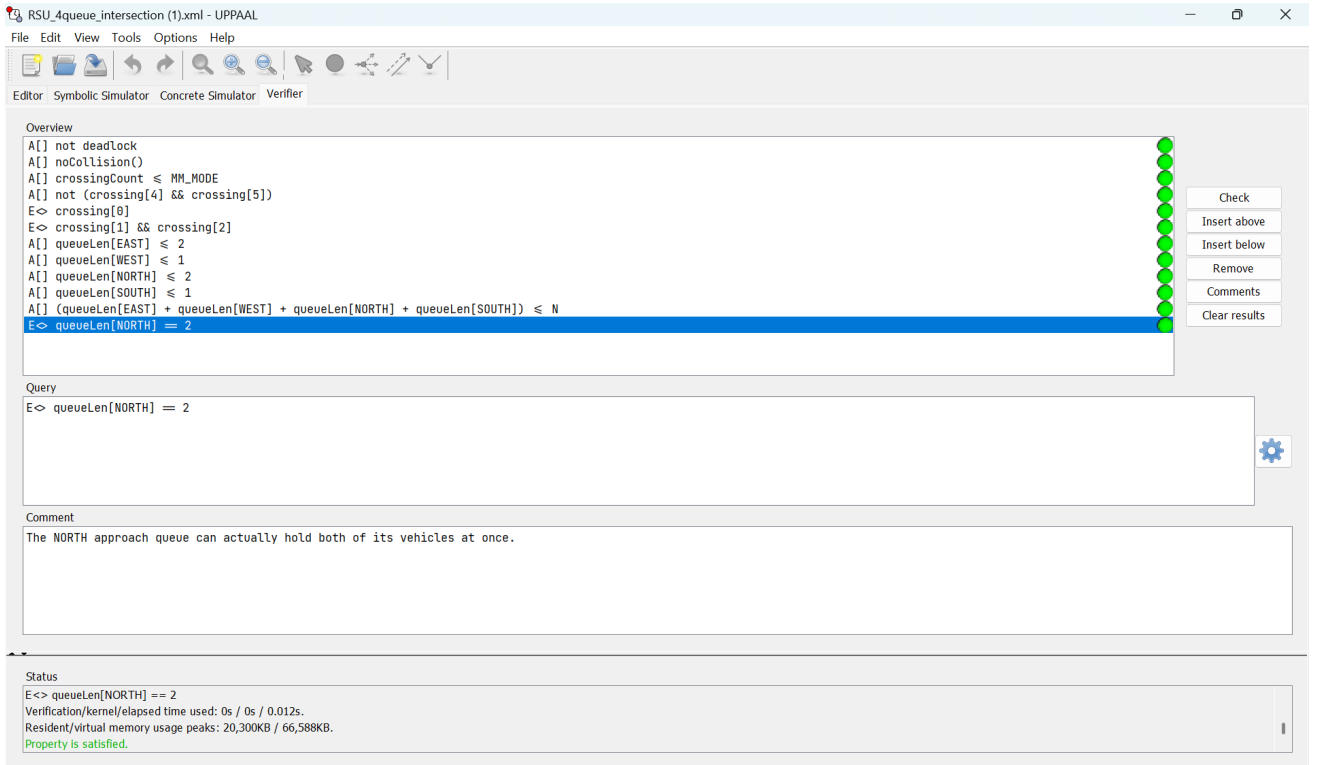


Fig. 7. Verifier result for the four-queue UPPAAL model. The checked properties include deadlock freedom, collision freedom, shared capacity for M/M/1 or M/M/2, critical V4/V5 mutual exclusion, emergency reachability, M/M/2 overlap reachability, and queue-bound checks for each approach.

Both models were verified against the same set of core properties, all of which hold: absence of deadlock, absence of collisions, respect of the capacity bound $\text{crossingCount} \leq \text{MM_MODE}$, mutual exclusion of the critical North-right/South-left manoeuvre, and reachability of both the emergency crossing and the simultaneous crossing of two non-conflicting main-road vehicles. The four-queue model additionally satisfies per-approach queue-length bounds, confirming that each of the four queues behaves correctly.

For the intersection-level safety and priority properties evaluated here, the two models are behaviourally equivalent. The four-queue model differs only by enforcing head-of-line

ordering within each approach and by making the queue structure explicit. The single-queue model yields a smaller state space and represents the intersection as one pooled resource, whereas the four-queue model more closely mirrors a realistic per-approach or per-lane controller at the cost of a slightly larger state space.

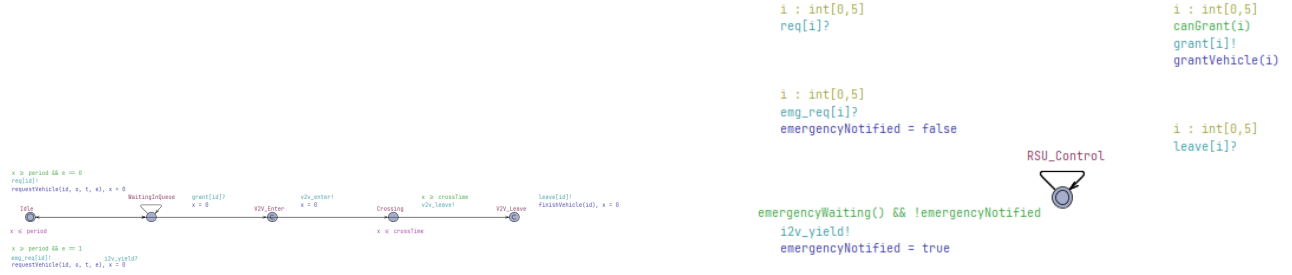


Fig. 8. UPPAAL single-queue model. The screenshots show the simpler abstract model in which the RSU controls one logical waiting structure and applies emergency priority, main-street priority, M/M/1 or M/M/2 capacity, and conflict checking before issuing a grant.

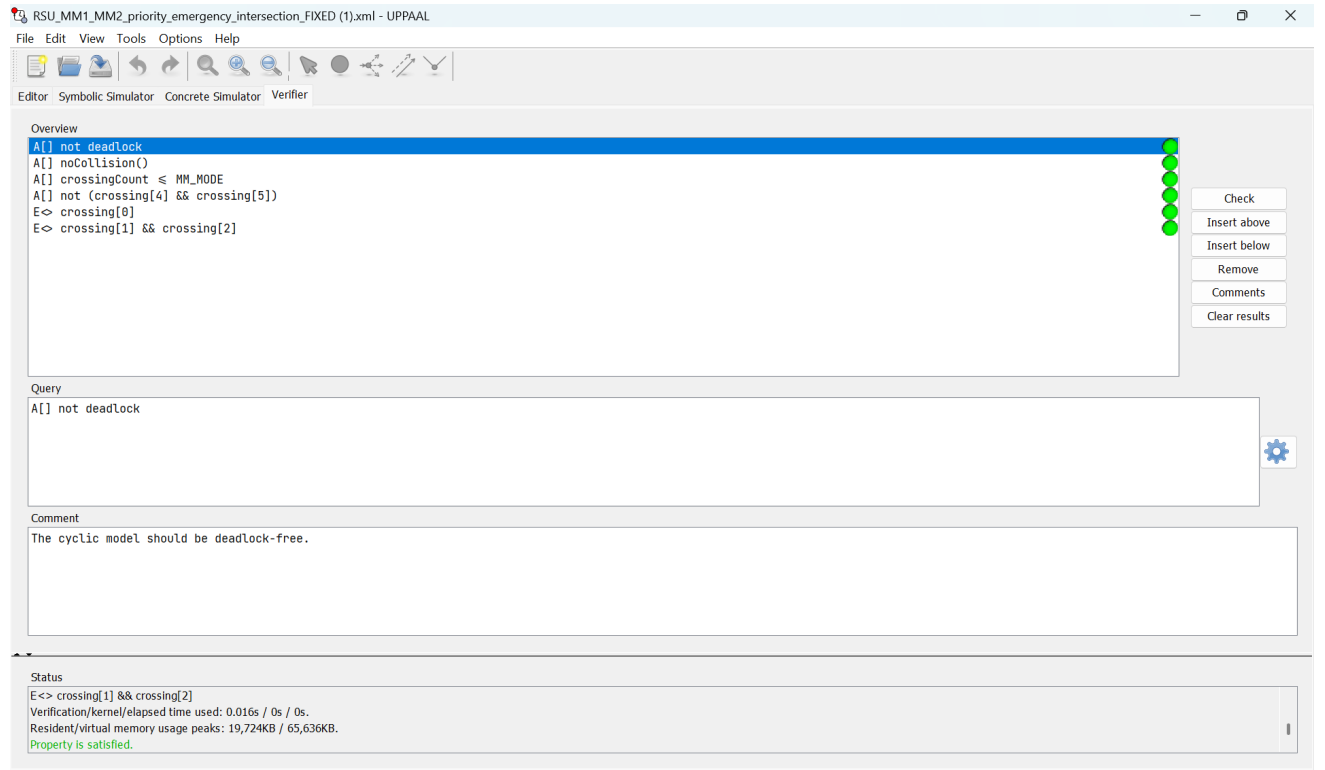


Fig. 9. Verifier result for the single-queue UPPAAL model. The checked properties show that the model is deadlock-free, collision-free, respects M/M/1 or M/M/2 capacity, prevents the critical V4/V5 conflict, allows the emergency vehicle to cross, and demonstrates M/M/2 overlap for two compatible main-road vehicles.

TABLE III
COMPARISON OF THE SINGLE-QUEUE AND FOUR-QUEUE UPPAAL MODELS

Aspect	Single-queue model	Four-queue model
Queue structure	One shared priority queue	Four per-approach queues, represented by <code>queueLen[R]</code> with $R = 4$
Service discipline	Global priority selection over all waiting vehicles	Per-approach head-of-line selection, then global arbitration between queue heads
Priority rules	Emergency > main road > side road; right-turn-before-left-turn; vehicle ID tie-break	Identical priority rules
Intersection capacity	Shared capacity; <code>crossingCount ≤ MM_MODE</code>	Identical shared capacity; capacity is not assigned per queue
Conflict handling	Intersection-wide conflict check	Identical intersection-wide conflict check
Core safety and reachability properties	All core properties hold	All core properties hold, plus per-approach queue-bound checks
State space	Smaller and simpler	Slightly larger because queue state is explicit for four approaches
Represents	One pooled intersection resource	Per-approach or per-lane controller feeding one shared crossing

The model was checked against the main safety and reachability queries listed in Table IV together with their run-time counterpart in the Week 3 deterministic C implementation. Because the UPPAAL declaration block is already written in a C-like action language, the mapping from model to code is structural. Every global array, helper function, conflict function, and grant function is copied under the same name. The vehicle locations become a per-vehicle state variable with a software clock, and the model’s non-deterministic selection over grantable vehicles is resolved deterministically by repeatedly granting the best-priority grantable candidate until none remains.

TABLE IV
MODEL PROPERTIES AND RUN-TIME CHECKS

Property	Query / run-time check
Deadlock freedom	$A[] \text{ not deadlock} - \text{simulation never stalls}$
Collision freedom	$A[] \text{ noCollision}() - \text{assert}() \text{ every tick}$
Capacity	$\text{crossingCount} \leq \text{MM_MODE} - \text{counter assertion}$
Critical conflict V4/V5	$A[] \text{ not}(\text{crossing}[4] \&\&\text{crossing}[5]) - \text{assertion every tick}$
Emergency reachability	$E<> \text{crossing}[0] - \text{V0 grant observed}$
M/M/2 overlap	$E<> \text{crossing}[1] \&\&\text{crossing}[2] - \text{V1 and V2 together, cross}=2$

VII. HW/SW CODESIGN PARTITION

Because the UPPAAL model and the C baseline already agree, the codesign partition is behaviour-preserving by construction. It changes where a function runs, not the verified safety contract. The partitioning rule is simple: regular, parallel, and timing-critical work moves to hardware, while irregular, control-heavy, and frequently revised work stays in software.

The scheduler loop, V2I/I2V/V2V message handling, state updates, emergency handling, and priority logic stay in software. Conflict checking is different: `conflict(a,b)`, `conflictsWithCrossing(i)`, and `noCollision()` are pure functions of the current state. Therefore, they are good candidates for hardware implementation. In hardware, all pairwise conflict checks can evaluate in parallel and return a bounded-time `safe_to_grant` result.

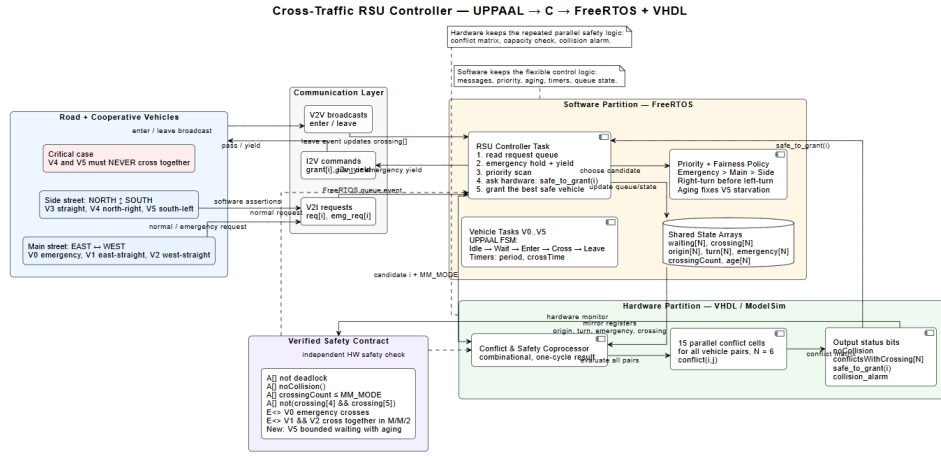


Fig. 10. Proposed hardware/software codesign partition. The FreeRTOS software partition keeps vehicle tasks, RSU control logic, queues, priority, and fairness policy. The VHDL hardware partition implements the combinational conflict and safety coprocessor.

TABLE V
HW/SW PARTITION MAPPING

Function / component	Reason
Vehicle FSM in software	Task-based behaviour and per-vehicle timers
RSU scheduler in software	Sequential and branch-heavy control
Request, grant, and leave queues in software	V2I, I2V, and V2V message handling
Emergency handling in software	Policy decision and I2V yield control
Priority and aging in software	Stateful and most frequently revised
<code>conflict(a,b)</code> in hardware	Static rule; pairwise checks can run in parallel
<code>conflictsWithCrossing(i)</code> in hardware	Candidate conflict scan becomes a vector check
<code>noCollision()</code> in hardware and software	Safety monitor and assertion check
<code>safe_to_grant(i)</code> from hardware to software	Capacity and conflict decision in one bit

TABLE VI
FREERTOS SOFTWARE COMPONENTS

Component	Implementation	Purpose
Vehicle task V0–V5	FreeRTOS task	Request, wait, cross, leave
RSU task	FreeRTOS task	Priority scan and grant decision
Request queue	FreeRTOS queue	V2I request and emergency request messages
Grant queue	FreeRTOS queue	I2V grant messages
Leave queue	FreeRTOS queue	Vehicle completion messages
Yield broadcast	Task notification	Emergency yield warning
Aging logic	Software counter	Bounded waiting for low-priority movements
HW interface	<code>hw_safe_to_grant(i)</code>	Register-mapped safety query

VIII. FREERTOS SOFTWARE ARCHITECTURE

The software side is organized as a FreeRTOS task set. One vehicle task is used for each vehicle V0–V5. The RSU controller task reads requests, applies emergency priority, checks the hardware safety output, and sends grant or wait messages. FreeRTOS queues carry request, grant, and leave messages.

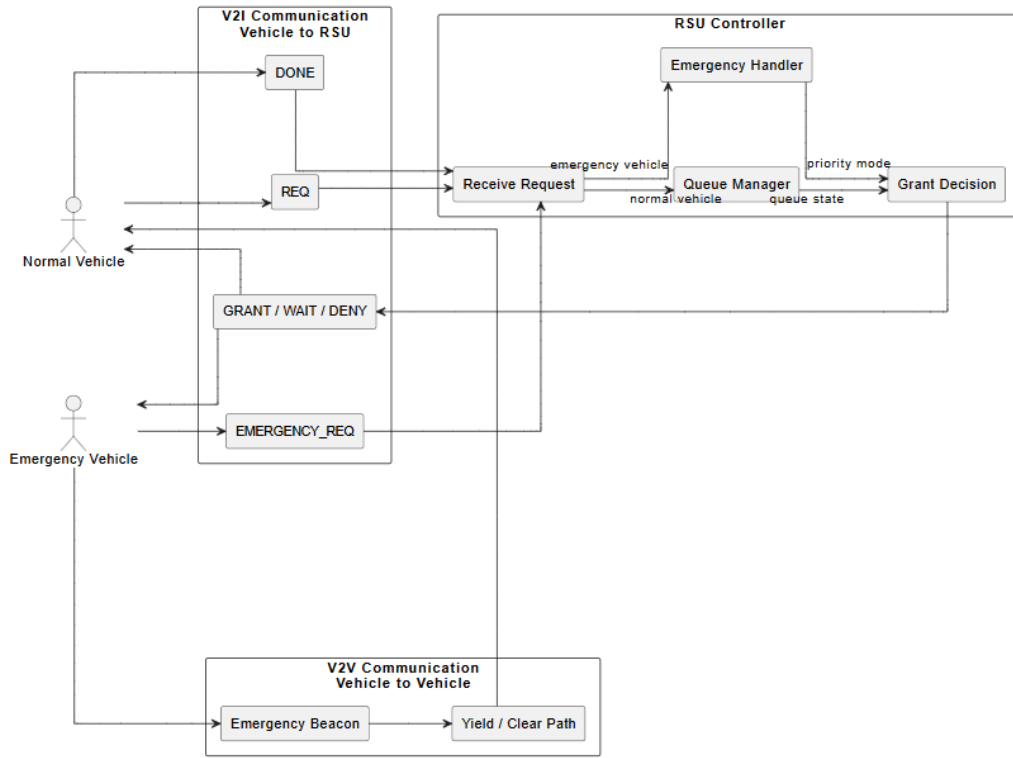


Fig. 11. V2I and V2V communication model. Vehicles send request and done messages to the RSU. The RSU replies with grant, wait, or deny messages. Emergency vehicles additionally broadcast a V2V beacon that triggers yield behaviour.

IX. EMERGENCY AND FAIRNESS HANDLING

While the emergency vehicle V0 is waiting, the controller blocks new grants to normal vehicles and broadcasts an I2V yield message. V0 is granted only when the intersection becomes safe. Emergency priority therefore changes the order of service, but it does not bypass the safety check.

The strict-priority baseline can starve the lowest-priority movement, especially V5. To reduce this risk, an aging counter can be used. The longer a vehicle waits, the more its effective priority improves. This fairness extension remains in software because it is stateful and policy-dependent. It only changes which safe vehicle is selected; it never overrides the hardware safety decision.

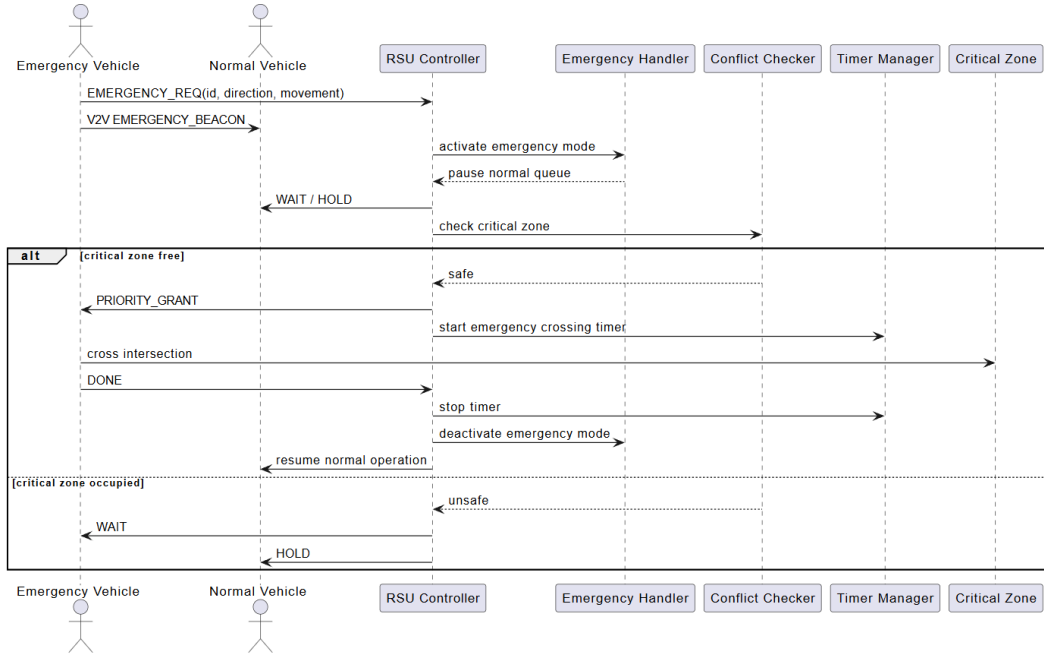


Fig. 12. Sequence of an emergency pre-emption. The emergency request and V2V beacon activate emergency mode, normal traffic yields, the RSU checks safety, grants the emergency vehicle, and then returns to normal scheduling.

X. HARDWARE SAFETY COPROCESSOR AND INTERFACE

The hardware side of the design is a combinational conflict and safety coprocessor. It receives the vehicle state, including origin, turn direction, emergency flag, and crossing flag. It checks whether any pair of currently crossing vehicles conflicts and whether a candidate vehicle is safe to grant.

TABLE VII
HW/SW INTERFACE REGISTERS

Register	Meaning
STATE_VEH[i] W, 6b	origin(2), turn(2), emergency(1), crossing(1)
MODE W, 1b	0 = M/M/1, 1 = M/M/2
CAND W, 3b	Candidate vehicle ID under test
CTRL W, 1b	Start or latch inputs
ST_SAFE R, 1b	safe_to_grant(CAND)
ST_NOCOL R, 1b	noCollision() over the fleet
ST_CWC R, Nb	conflictsWithCrossing[i] vector
ST_ALARM R/IRQ, 1b	Latched collision alarm

XI. TEST AND VALIDATION METHODOLOGY

Verification proceeds bottom-up. First, the UPPAAL model is checked. Then the deterministic C baseline is executed with run-time assertions. Finally, the FreeRTOS and hardware components are tested against the same safety and reachability expectations.

TABLE VIII
TEST CASES AND STATUS

ID	Test case / expected result	Status
T1	M/M/1: crossingCount never > 1	Pass C
T2	M/M/2: V1 and V2 may cross together	Pass C
T3	Emergency V0 gets highest priority	Pass C
T4	I2V yield broadcast appears	Pass C
T5	V4/V5 never granted together	Pass C + UPPAAL
T6	Fairness: V5 eventually granted	Planned
T7	No collision-assertion failure	Pass C
T8	No hardware collision alarm appears	Planned

XII. RESULTS AND DISCUSSION

A. M/M/1 Result

In M/M/1 mode, only one vehicle is granted at a time. The value of crossingCount never reaches 2, which confirms single-server behaviour.

=1	[t= 6] RSU GRANT -> V1 (EAST STRAIGHT) cross
	[t= 8] V2 REQ (WEST STRAIGHT) -> RSU <--
queued	
	[t=11] V1 DONE -> Idle cross=0
	[t=12] RSU GRANT -> V2 (WEST STRAIGHT) cross
=1	

Listing 1. M/M/1 log excerpt.

B. M/M/2 Result

In M/M/2 mode, V1 and V2 can be granted together because their movements are non-conflicting. The value of crossingCount reaches 2, proving that the second service slot is used safely.

	[t= 6] V1 REQ (EAST STRAIGHT) -> RSU
	[t= 6] RSU GRANT -> V1 (EAST STRAIGHT) cross
=1	
	[t= 8] V2 REQ (WEST STRAIGHT) -> RSU
	[t= 8] RSU GRANT -> V2 (WEST STRAIGHT) cross
=2	<-- M/M/2
	[t=11] V1 DONE -> Idle cross=1
	[t=12] V4 REQ (NORTH RIGHT) -> RSU
	[t=12] V5 REQ (SOUTH LEFT) -> RSU
	[t=14] RSU GRANT -> V4 (NORTH RIGHT) cross=1
	<-- V4 only

Listing 2. M/M/2 log excerpt.

C. Quantitative Comparison of M/M/1 and M/M/2

To quantify the benefit of the second service slot, waiting and crossing-completion times were extracted directly from the deterministic run logs of Listings 1 and 2, using the arrival periods and crossing times of Table II. All values are given in the abstract time units of the model. Table IX reports the results for the non-conflicting main-road pair V1 and V2, which is the pair eligible for parallel crossing in M/M/2 mode. The waiting time of a vehicle is measured from its request to its grant, and the makespan is the time at which the last of the two vehicles finishes crossing.

In M/M/1 mode, V2 must wait for V1 to leave the intersection before it can be granted, giving a mean waiting time of 2.0 time units for the pair and a makespan of 17. In

TABLE IX
QUANTITATIVE COMPARISON OF M/M/1 AND M/M/2 FOR THE
MAIN-ROAD PAIR (V1, V2), IN MODEL TIME UNITS

Metric	M/M/1	M/M/2
V1 waiting time	0	0
V2 waiting time	4	0
Mean waiting time (V1, V2)	2.0	0.0
Max concurrent crossings	1	2
Makespan to clear V1 and V2	17	13

TABLE X
FINAL VALIDATION SUMMARY

Feature	Evidence	Status
M/M/1 serial crossing	crossingCount ≤ 1	Pass
M/M/2 parallel crossing	V1 and V2 together	Pass
Emergency priority	V0 exclusive grant	Pass
I2V yield broadcast	Observed during emergency request	Pass
V4/V5 mutual exclusion	0 joint-crossing violations	Pass
Software safety assertions	0 failures	Pass
Fairness with aging	Starvation fix proposed	Planned
HW collision alarm	Proposed hardware monitor	Planned

M/M/2 mode, V1 and V2 are granted in parallel because their straight movements from opposite approaches do not conflict; both waiting times fall to zero and the pair clears at $t = 13$, a reduction of 4 time units (about 24%) in makespan. This quantifies the throughput benefit of the pooled second server. Crucially, the improvement does not weaken safety: in the same M/M/2 run the critical pair V4 and V5 is still correctly serialised, with V4 granted alone at $t = 14$ (waiting time 2) and V5 held back, so the second slot is used only for non-conflicting movements.

D. Emergency Result

V0 is queued with highest priority. The RSU broadcasts an I2V yield message, waits until the intersection is safe, and then grants V0 exclusively. This confirms emergency pre-emption without violating the conflict rules.

E. Safety Result

Across the tested runs, the collision condition never fails. The model keeps crossingCount within the selected mode and never grants the conflicting V4/V5 pair together.

XIII. LIMITATIONS AND FUTURE WORK

The current results are based on the deterministic C baseline and UPPAAL verification. The quantitative comparison is drawn from a single illustrative scenario rather than a statistical sweep of arrival rates, so the reported waiting times and makespan should be read as indicative of the M/M/2 benefit rather than as steady-state averages. The FreeRTOS implementation and VHDL safety coprocessor are planned as the next implementation step. Future work includes hardware simulation in ModelSim, end-to-end timing measurement, a statistical evaluation of waiting time and throughput over randomised arrival patterns, fairness verification with aging, and deployment on FPGA or PCB hardware.

XIV. CONCLUSION

This paper presented an RSU-based autonomous intersection controller from requirements to UPPAAL verification, C baseline implementation, and hardware/software codesign proposal. The UPPAAL model shows that the system is deadlock-free, collision-free, respects M/M/1 and M/M/2 capacity, supports emergency priority, and allows safe parallel crossing for non-conflicting movements. A quantitative comparison of the two service modes shows that M/M/2 removes the waiting time of the non-conflicting main-road pair and shortens the time to clear it by about 24% in the studied scenario, without weakening the conflict guarantees. The comparison between the single-queue and four-queue models shows that both preserve the same safety behaviour, while the four-queue model represents a more realistic per-approach controller.

AI USAGE DISCLOSURE

Portions of the requirements interpretation, UPPAAL comparison, HW/SW partitioning discussion, interface design, quantitative analysis, and verification planning in this paper were developed with AI assistance.

REFERENCES

- [1] Team A – Group 01, “Autonomous Intersection RSU Controller: Functional and Non-Functional Requirements,” internal project document, Hochschule Hamm-Lippstadt, 2026.
- [2] Team A – Group 01, “RSU M/M/1 and M/M/2 priority emergency intersection: UPPAAL Timed-Automata Model,” internal project artefact, Hochschule Hamm-Lippstadt, 2026.
- [3] Team A – Group 01, “Cross-Traffic Intersection Management for Autonomous Vehicles – Week 3: First Abstract Implementation,” internal seminar report, Hochschule Hamm-Lippstadt, 2026.
- [4] Team A – Group 01, “Hardware/Software Codesign Proposal: Cross-Traffic Intersection Management for Autonomous Vehicles,” Week 4 milestone report, Hochschule Hamm-Lippstadt, 2026.
- [5] Real Time Engineers Ltd., *The FreeRTOS Reference Manual*, accessed 2026.
- [6] K. Dresner and P. Stone, “A Multiagent Approach to Autonomous Intersection Management,” *Journal of Artificial Intelligence Research*, vol. 31, pp. 591–656, 2008.
- [7] L. Chen and C. Englund, “Cooperative Intersection Management: A Survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 2, pp. 570–586, 2016.
- [8] R. Alur and D. L. Dill, “A Theory of Timed Automata,” *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.
- [9] G. Behrmann, A. David, and K. G. Larsen, “A Tutorial on Uppaal,” in *Formal Methods for the Design of Real-Time Systems (SFM-RT 2004)*, LNCS vol. 3185, Springer, 2004, pp. 200–236.